



Коллектив исслед...

Искусственный Интеллект +

ТЕХНОЛОГИЯ +

Программирование +

Машинное обучение +

Наука о данных +

Что такое чанкинг?

Вы устанавливаете верхний предел качества RAG при приеме данных, до выполнения первого запроса

12 минут чтения · 29 июля 2026 года



Paolo Perrone



Слушай



Поделиться

Разбиение на фрагменты позволяет разделить документы на более мелкие части перед тем, как система RAG внедрит их и проиндексирует, чтобы при выполнении запроса извлекался только тот фрагмент, который на него отвечает. Разбиение на фрагменты ограничивает качество ответов: размер фрагмента, границы и перекрытие определяют, что модель вообще может увидеть, и ни на одном последующем этапе нельзя восстановить то, что было отброшено из-за неудачного разбиения.

Часть 11 курса RAG & Search

TL;DR

- **Определение:** фрагмент — это несколько сотен текстовых токенов, встроенных и проиндексированных как отдельная извлекаемая единица.
- **Почему это важно:** при ранжировании извлекаются только существующие фрагменты. Политика, разделенная на два фрагмента, не дает ответа на вопрос.

- **Размер и границы:** слишком маленькие фрагменты лишают отрывок необходимого контекста, а слишком большие усредняют несвязанные отрывки в один вектор, который не соответствует ни одному конкретному вопросу. Оптимальные границы — абзацы, заголовки и окончания предложений.
- **Перекрытие:** конец каждого фрагмента копируется в начало следующего, поэтому предложение, которое пересекает границу фрагмента, сохраняется целиком хотя бы в одном фрагменте. Повторы требуют дополнительных векторов, поэтому перекрытие должно составлять от 10 до 20 процентов от длины фрагмента.
- **Подвох:** сложные стратегии нарезки почти не выигрывают у простых, а в одном из тестов простая стратегия одержала безоговорочную победу. Прежде чем прибегать к чему-то экзотическому, настройте базовые параметры.

До разбиения на фрагменты — по одному вектору на документ

В самой простой версии конвейера нет сплиттера. У вас есть папка с PDF-файлами компании, модель встраивания и векторная база данных. Вы встраиваете каждый документ как один вектор, сохраняете по одной строке на документ и сопоставляете их с запросами. Это индексация на уровне документов. Для небольших файлов на одну тему она работает. Мы разобрали весь конвейер в статье [Что такое RAG?](#)

Эмбеddинг сжимает отрывок текста в один вектор фиксированной длины, который располагается рядом с текстами со схожим смыслом. [Что такое эмбеddинги?](#) разбираем механизм. Один вектор на документ означает один вектор на весь 80-страничный справочник для сотрудников.

Ни одна встраиваемая модель не может прочитать 80 страниц за один проход, поэтому конвейер встраивает руководство по частям и усредняет их в один вектор документа. Адаптация, расходы, обучение технике безопасности и правила предоставления отпусков — все это относится к одной категории.

Почему метод «один вектор на документ» не работает

Новый сотрудник спрашивает у корпоративного бота, сколько дней отпуска он получит после третьего года работы. Ответ находится на 41-й странице

руководства. При извлечении данных вопрос сравнивается с одним вектором на документ, и возвращается наиболее близкий вариант.

Вектор руководства представляет собой среднее значение по всем темам в книге, а среднее значение по далеким друг от друга понятиям плохо соотносится с конкретным вопросом.

На странице 12 руководства сказано:

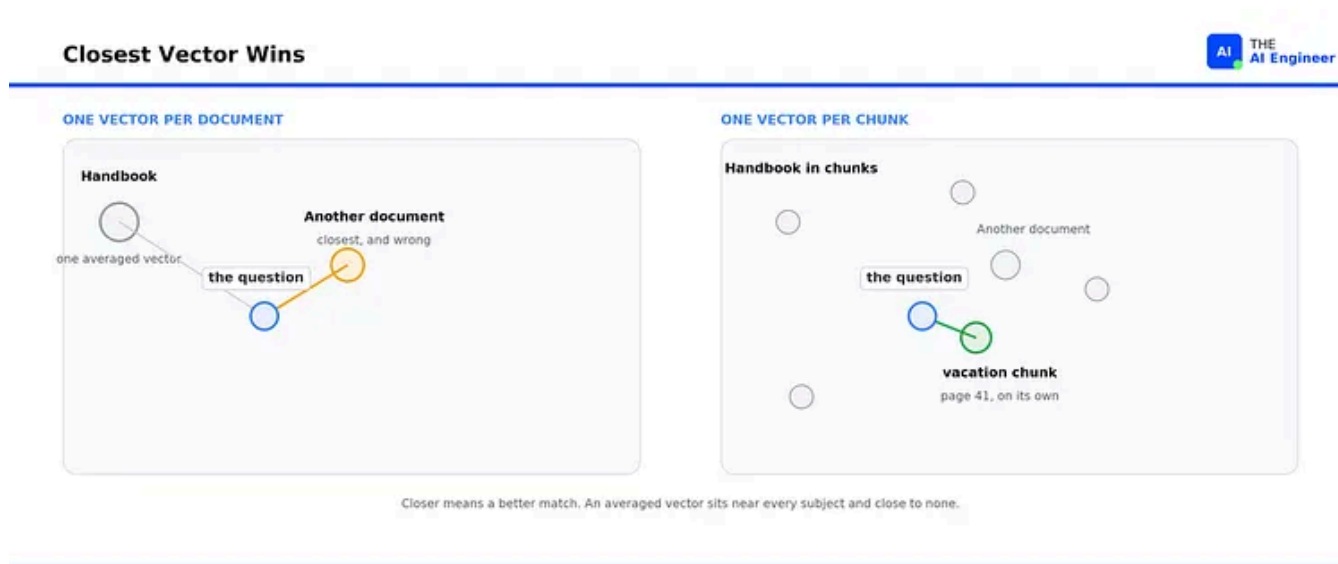
Отчеты о расходах на сумму свыше 500 долларов США требуют утверждения директором перед отправкой.

На странице 41 написано:

Сотрудникам со стажем работы от трех лет ежегодно предоставляется 20 дней оплачиваемого отпуска.

Оба вопроса вместе с другими семьюдесятью восемью страницами вводятся в модель эмбединга и преобразуются в один список чисел. Именно с этим списком сравнивается вопрос об отпуске.

The handbook ranks below documents narrow enough to match the question, so the bot answers the new hire without ever seeing page 41.



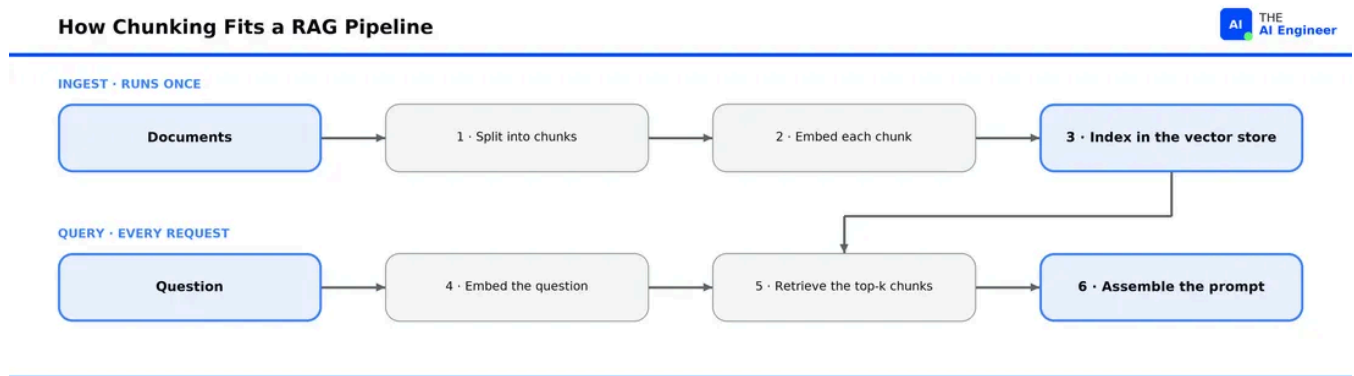
How Chunking Actually Works

A chunk has two jobs at once. It has to be small enough that its vector stays specific. It also has to be complete enough that the passage answers the question by itself.

Those two pull against each other, and three decisions settle where you land between them:

The pipeline runs in two halves. Ingest happens once:

1. A splitter cuts each document into chunks.
2. The vectors land in a vector store with the chunk text attached. We compared the stores themselves in [Vector Database Showdown](#).
3. how much to repeat across cuts.



Query time runs the same steps from the other side:

1. The same model embeds the question.
2. The index returns the top-k closest chunks, 3 to 10 in most setups.
3. The pipeline pastes those chunks into the prompt as the context the model answers from.

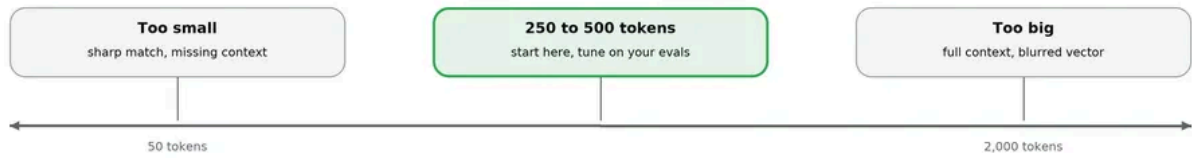
Every stage downstream of step 1 handles chunks. The document as a whole never appears again.

(1) **Where to cut** is which splitter you run, and the options runs from dumb to expensive:

1. **Fixed-size splitting** cuts every N tokens no matter what the text is doing, fast and uniform. It has no notion of where a sentence or a rule ends, so a cut lands wherever the count runs out.

2. **Recursive splitting** cuts where the writing already pauses. It tries paragraph breaks first, then feeds any piece that is still too big back through itself on line breaks, then sentence ends. Running on its own output is what makes it recursive.
3. **Structure-based splitting** follows the document's own skeleton: markdown headings, HTML sections, PDF pages. The author grouped related content when they wrote it, and the split inherits that grouping, so tables and code blocks survive whole.
4. **Semantic chunking** embeds every sentence, then compares each one against the sentence before it. A large jump between two neighbours means the subject changed, so the splitter cuts at that point. The cost is an embedding pass over every sentence at ingest.
5. **Agentic chunking** hands the document to an LLM and lets it choose the boundaries, the way a person marking up a manual would. It is the only strategy that can tell a heading from a caption from a footnote, at a cost of one model call per document.
6. **Contextual enrichment** does not move the cuts at all. After splitting, a model writes a one-line description per chunk, naming the document and section it came from, and that line goes in front of the chunk before embedding. A chunk that begins mid-rule then carries the policy name its own text left out.

(2) **How big to cut** is the size setting, and the working band is 250 to 500 tokens. Tune from there against your own queries. A token-level evaluation by Chroma ran nine chunker configurations over identical corpora, models, and queries and found an 8-point recall spread between best and worst. Recall here is the share of each needed passage's tokens that lands in the retrieved set. Plain recursive splitting at 400 tokens scored 89.5 percent. The best semantic chunker cleared that by under two points. The worst one sat at the bottom of the table at 83.6. OpenAI's popular 800-token default gave up a point to recursive. Between the best cut and the worst, the share of each needed passage that never reaches the model doubles, from about 8 percent missing to about 16 . Changing a splitter setting costs one re-index, which makes this the cheapest 8 points of recall in the RAG pipeline.



(3) **How much to repeat** is the overlap setting. A fixed-size splitter on the handbook ends chunk 12 mid-rule:

...Employees with three or more years of service accrue

Chunk 13 opens with the payoff:

twenty days of paid vacation annually, subject to manager approval...

Retrieval pulls chunk 13, since that is where “twenty days” and “vacation” live, and the model quotes the number with no eligibility rule attached. Overlap is the fix. The splitter copies the last 40 to 60 tokens of each chunk onto the front of the next, so a sentence that crosses a cut survives whole in at least one piece.

Where the Cut Lands Decides What Survives

FIXED-SIZE CUT, NO OVERLAP



SAME CUT WITH OVERLAP



The repetition has a price. Fifteen percent overlap means fifteen percent more vectors to embed, store, and search. The duplicated text can also return twice, so two of your ten retrieved slots hold the same sentences and the model sees less of the corpus than you think.

The working baseline fits in one call:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",
    chunk_size=400,
    chunk_overlap=60,
)
chunks = splitter.split_text(handbook_text)
```

The plain `RecursiveCharacterTextSplitter` constructor measures `chunk_size` in characters rather than tokens, so asking for 400 gets you about 100 tokens of text. Building it through `from_tiktoken_encoder` measures in real tokens instead. This is the first thing I check when a pipeline is retrieving fragments.

Who's Actually Building With This

Anthropic tested the ingest-side fix at scale. Their embeddings-only baseline failed to surface the needed chunk in the top 20 results 5.7 percent of the time . Contextual enrichment cut that to 3.7 percent, a 35 percent drop bought entirely at ingest, with no change to the model or the query. Two retrieval levers on top took it to 1.9 percent. The first is BM25, the classic keyword scorer we unpacked in [What is Semantic Search?](#). The second is a reranker, which re-scores the shortlist before the top results ship.

NVIDIA benchmarked chunking strategies across five document sets, and the winner was page-level chunking: one chunk per PDF page, with no splitting logic at all. It averaged 0.648 accuracy, where 1.0 is a perfect score, beating every other strategy on average with the lowest variance across document types . A page break rarely lands mid-table or mid-clause, which is most of what a splitter is trying to avoid.

A **Hugging Face** team benchmarking RAG over nuclear engineering documents got the opposite result: the purpose-built chunker lost to the default one . Their baseline was LangChain's recursive splitter at 2,000 characters with a 400-character overlap, one of several sizes they tried, and the size barely moved the result . Across 156 test queries it beat their section-aware chunker 70.5 percent to 63.8, scoring the share of queries whose source document landed in the ten retrieved results. Their reading of why: cutting strictly on section headers orphaned content that ran across a boundary. The overlap in the default setup kept that same content together.

What Can Go Wrong (and What's Overhyped)

Five failure modes account for most chunking pain:

1. **Ingest-time model bills.** Semantic chunking runs an embedding pass over every sentence in the corpus. Agentic chunking runs a model call per document.
2. **Near-duplicate retrieval.** Generous overlap plus repetitive sections fills the top-k list with copies of the same paragraph. Every duplicate takes a slot, so the passages sitting just below the cutoff get pushed out and the model sees the same fact three times instead. Cap overlap and deduplicate retrieved chunks by text similarity.
3. **The precision trap.** Shrinking chunks makes each vector more specific and strips away the context the passage needed to answer the question. Cut the handbook fine enough and the vacation number lands in a chunk of its own, with the years-of-service condition sitting in the chunk above it. Retrieval finds the number and the model answers without the rule.
4. **Ingest-time model bills.** Semantic chunking runs an embedding pass over every sentence in the corpus. Agentic chunking runs a model call per document. On a million documents those passes cost real compute before the first query ships. Contextual enrichment runs a call per chunk and is the exception, because prompt caching drops it to about a dollar per million document tokens.
5. **Silent truncation.** Every embedding model has a maximum input length, and it drops whatever runs past that point without raising an error. all-MiniLM-L6-v2, pulled about 254 million times a month, reads 256 tokens. Feed it the 400-token chunks the baseline above produces and it embeds the opening stretch, drops the rest, and returns a vector that looks exactly like any other. Its 256 limit counts MiniLM's own word pieces, which are not the tokens your splitter counted⁵. Check your model's limit before you set chunk size.

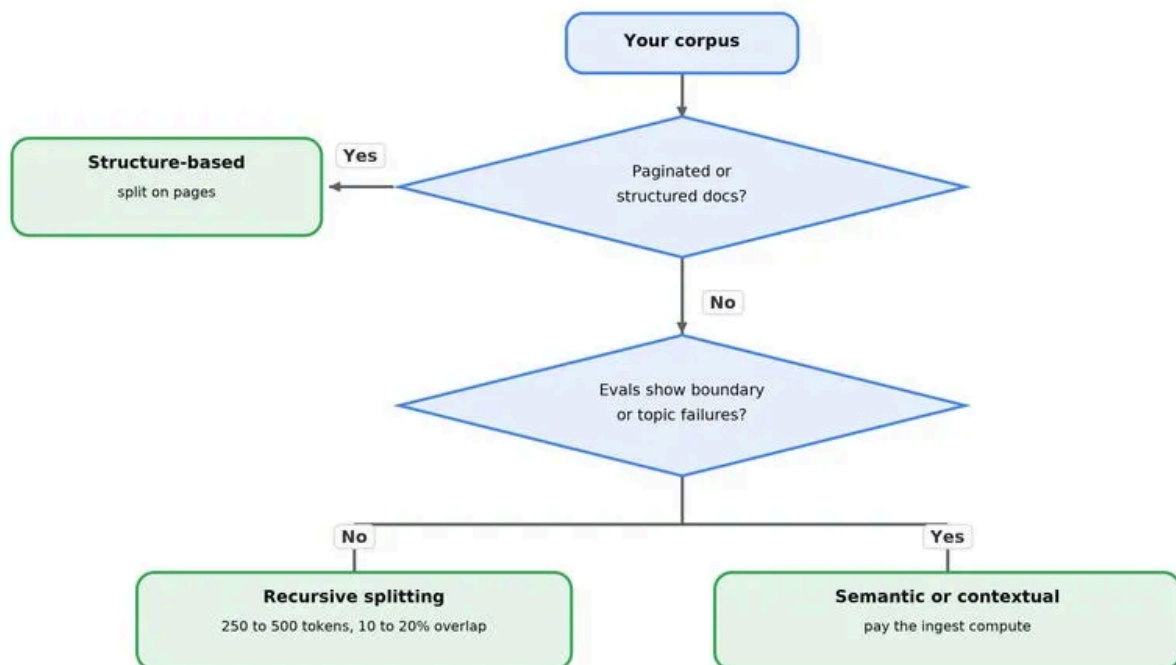
The loudest claim about chunking right now is that long-context models killed it. Anthropic's own guidance puts the no-RAG threshold at 200,000 tokens, about 500 pages. Below it, keep the corpus in the prompt and let prompt caching cover the rereads. Above it, you are retrieving, and retrieval runs on chunks. The other hype magnet is the exotic chunker, and the measured gap between the fanciest one and tuned recursive splitting was two recall points.

 **Engineering Lesson:** No single chunking setup is right for every corpus, and you cannot reason your way to yours. It depends on how much structure your documents carry, how narrow your queries run, and how much ingest compute you can spend. Keep fifty real queries with their known source passages, re-run them after every change, and read the number. A two-point gain and random noise look identical until you do.

Which of the six you land on comes down to two questions:

1. The first is **what your documents look like**. Paginated files mean page-level splitting, which NVIDIA found beats every strategy on average. Headings are less reliable, since the Hugging Face team cut strictly on them and lost. Everything else starts recursive at 400 tokens with 60 of overlap.
2. The second is **what your eval set says**. Misses that trace to a cut landing mid-rule or mid-topic are what justify semantic or agentic chunking, or a contextual enrichment pass over the chunks you already have. Misses that trace anywhere else mean the splitter is not your bottleneck and the next fix sits further down the pipeline.

Pick Your Chunking Strategy



Under 200k tokens: skip retrieval, keep the whole corpus in the prompt

The One Thing to Remember

A RAG system assembles every answer from pieces it cut before the first question arrived. Tuning retrieval, swapping embedding models, and adding rerankers all operate downstream of the same ceiling: the retriever chooses among the chunks the splitter made, and the model reads only what the retriever returns. Ingestion is where an answer becomes findable, or stops existing.

💬 *What did a bad chunk boundary cost you? A wrong number, a split table, a policy quoted without its condition: tell me in the comments. I read every one.*

Where to Next?

- 📖 **Go deeper:** [How Perplexity Built Their Search Engine](#), what retrieval looks like when the corpus is the whole web.
- 🔄 **Related:** [Agentic RAG vs CUA vs A2A](#), what happens when the retrieval loop itself becomes an agent.
- 🔗 **Prerequisite:** [How DoorDash Built Their RAG System](#), the guardrail side that catches bad answers after generation.

FAQ

What is chunking in RAG?

Chunking cuts each document into passages of a few hundred tokens, and the RAG pipeline embeds and indexes those passages instead of the whole file. Retrieval then ranks passages, so the model receives the paragraph that answers the question rather than everything around it. Size, boundaries, and overlap decide whether that passage arrives focused and complete.

What chunk size should I use for RAG?

Start between 250 and 500 tokens with 10 to 20 percent overlap, then tune upward against your own queries. The starting band is deliberately below where benchmarks land, since small chunks fail loudly and big ones fail quietly. NVIDIA's five-dataset benchmark found 512 to 1,024 tokens best for token-based splitting,

with page-level chunking the most consistent overall and 128-token chunks worst. Cut too small and a chunk keeps the number while losing its condition. Cut too big and its vector stops matching any specific question.

What is chunk overlap and how much should I use?

Overlap repeats the tail of each chunk at the head of the next, so sentences that straddle a boundary survive whole in at least one piece. The working range is 10 to 20 percent of chunk size. More overlap means more vectors to store and search, plus near-duplicate passages competing for top-k slots, so raise it only when boundary failures show up in your evals.

Do long-context models make chunking obsolete?

No. Anthropic's guidance puts the crossover near 200,000 tokens, about 500 pages. A corpus smaller than that can skip retrieval: it sits in the prompt, and prompt caching keeps rereading it cheap. Anything larger still needs retrieval, and retrieval operates on chunks.

Is semantic chunking better than fixed-size chunking?

Yes on some corpora, and by less than the name promises. A token-level benchmark measured the best semantic chunkers about two recall points above tuned recursive splitting, and the worst semantic chunker scored below plain recursive. A Hugging Face team found the stock recursive splitter beat section-aware chunking by almost seven points on nuclear engineering documents.

Originally published at <https://theaiengineer.substack.com> on July 29, 2026.

Artificial Intelligence +

Technology +

Programming +

Machine Learning +

Data Science +



Published in Data Science Collective

945K followers · Last published 3 days ago

Advice, insights, and ideas from the Medium data science community



Written by Paolo Perrone

13.6K followers · 172 following

Shipping Production AI: Agents, Inference, GPU. Read by 1M+ AI engineers. Founding Editor Data Science Collective <https://medium.com/data-science-collective>
